

- Thought for 13s >
- 以下是针对你4个问题的完整分析。

问题1: config.cmake 宏如何影响编译, 与 CMakeLists.txt 什么关系

传递链如下:

```
fawhq_e001_10_config.cmake
| doption(CONFIG_X_DOM_SOMEIP_RT_UDS_V4 ON)
| → 等同于 set(CONFIG_X_DOM_SOMEIP_RT_UDS_V4 ON)
|
|
|
|
|
| consys_bf.cmake (product_build_marco5, 第189行)
| if(CONFIG_X_DOM_SOMEIP_RT_UDS_V4)
|   set(product_static_lib_list_uds_v4 ${...})
|   x_dom_someip_rt_gen_lib_uds_${PRODUCT_NAME}_v4)
|
|
|
|
|
| proxy/CMakeLists.txt (第4行)
| add_subdirectory(x_dom_someip_rt_gen_uds)
|
|
|
|
|
| x_dom_someip_rt_gen_uds/CMakeLists.txt
| aux_source_directory(${CMAKE_CURRENT_SOURCE_DIR} ...)
| add_library(x_dom_someip_rt_gen_lib_uds_${PRODUCT_NAME}_v4 STATIC ...)
| → 编译 UDS 自己的 SOME/IP 生成代码 (协议编解码+信号接口)
|
|
|
|
|
| consys_bf.cmake (第237行)
| target_link_libraries(uds_v4 PRIVATE -Wl,--whole-archive
|   ${product_static_lib_list_uds_v4} → 包含上面的静态库
|   autosar_uds_${PRODUCT_NAME}_v4
|   -Wl,--no-whole-archive)
```

doption() 的本质 (consys_utility.cmake:93) :

```
macro(doption dfeature dflag)
| set(${dfeature} ${dflag}) → # 就是普通的 set()
endmacro(doption)
```

就是个 CMake 变量。所以 CONFIG_X_DOM_SOMEIP_RT_UDS_V4 是一个普通 CMake 布尔变量, 被 if() 判断来条件编译。

当前 fawhq_e001_10 的 UDS v4 实际生效的开关

CONFIG_X_DOM_CAN_RT_UDS_V4	OFF	→ CAN comif 不参与
CONFIG_X_DOM_SOMEIP_RT_UDS_V4	ON	→ 仅 SOME/IP comif 参与
CONFIG_SIP_AUTOSAR_UDS_V4	ON	→ AUTOSAR 诊断协议栈
CONFIG_GE_DIAGPROXY	ON	→ 诊断代理层
CONFIG_APP_UDSONIP	ON	→ DoIP 通信

- Update Todos
 - ☒ 追溯 config.cmake 宏 → CMakeLists.txt 的传递链
 - ☒ 分析 libuds_v4.so 的完整 SO 依赖关系
 - ☐ 分析 CAN 信号接入 UDS 的依赖引入方式
 - ☐ 梳理 comif ↔ COM 链路的完整调用链

- Thought for 1s >
- 问题2: libuds_v4.so 的完整依赖关系

编译产物结构

libuds_v4.so 通过 --whole-archive 把多个静态库“打包”成一个 .so, 没有对 lib*_com.so 的动态链接依赖。具体组成:

```
libuds_v4.so
| target_link_libraries(uds_v4 PRIVATE
|   -Wl,--whole-archive
|   |
|   | ${product_static_lib_list_uds_v4}
|   |   ├── ${DSAR_APP_LIB_CDD} (CDD通信驱动)
|   |   ├── dcos_dcms (DCMS中间件)
|   |   ├── ${UDSONIP_TARGET_NAME}_v4 (DoIP协议栈)
|   |   ├── diagproxy_ge_fawhq_e001_10 (诊断代理)
|   |   └── x_dom_someip_rt_gen_lib_uds_fawhq_e001_10_v4
|   |       └── (SOME/IP协议编解码 + 信号接口 get/set)
|   |
|   |
|   | autosar_uds_${PRODUCT_NAME}_v4 (AUTOSAR UDS栈)
|   |
```

```

-Wl,--no-whole-archive

PRIVATE 动态链接:
├── mini_dcos (DCOS运行时)
└── ${DCOS_LIBRARIES} (DCOS扩展库)

【不依赖】libfawhq_e001_10_com.so
```

三个 .so 的静态库组成对比

静态库	lib*_com.so	libuds_v4.so	lib*_uds_v6.so
DSAR_APP_LIB_CDD	✓	✓	✓
dcos_dcms	✓	✓	✓
x_dom_can_rt_gen_lib_com	✓	-	-
x_dom_someip_rt_gen_lib_com	✓	-	-
x_dom_can_rt_gen_lib_uds_v4	-	OFF	-
x_dom_someip_rt_gen_lib_uds_v4	-	✓	-
x_dom_can_rt_gen_lib_uds_v6	-	-	OFF
autosar_com	✓	-	-
autosar_uds_v4	-	✓	-
autosar_uds_v6	-	-	(if ON)
udsonip_v4	-	✓	-
diagproxy_ge	-	✓	-

关键结论：

- libuds_v4.so 编译时不依赖 lib*_com.so (没有 target link libraries 到它)
- 运行时也不依赖 — 两个进程通过 DCMS topic 通信，是松耦合的 IPC
- DiagProxy 虽然 include 了 x_dom_can_rt_gen_uds 和 x_dom_someip_rt_gen_uds 的头文件 (编译期头文件依赖)，但不链接 COM 的库

Update Todos

- ☒ 追溯 config.cmake 宏 → CMakeLists.txt 的传递链
- ☒ 分析 libuds_v4.so 的完整 SO 依赖关系
- ☒ 分析 CAN 信号接入 UDS 的依赖引入方式
- ☐ 梳理 comif ↔ COM 链路的完整调用链

Thought for 1s >

问题3：如果后续 APP 侧需要接入 CAN，如何引入依赖

以 fawhq_e001_10 为例，分三步：

步骤1: 修改 comif JSON，添加 UDS 的信号路由

在 x_dom_can_rt_fawhq_e001_10.json 的 dji_uds_service 路由节点中填信号：

```
// 当前是空的:
"dji_uds_service": {
  "read_mcu_rx_sigs_cfg": [], // ← 填：UDS需要读的MCU上行信号
  "read_mcu_tx_sigs_cfg": [], // ← 填：UDS需要读的SOC下行信号
  "write_to_mcu_sigs_cfg": [], // ← 填：UDS需要写到MCU的信号
  ...
}

// 改为例如:
"dji_uds_service": {
  "read_mcu_rx_sigs_cfg": [
    { "msg_name": "VDC_Config", "msg_id": "0x2b0", "channel": 1, "signals": [
      { "sig_name": "VDC_PowerMode", "start_bit": 12, "len": 2 },
      { "sig_name": "VDC_OTAMode", "start_bit": 14, "len": 2 }
    ] }
  ],
  "write_to_mcu_sigs_cfg": [
    // UDS需要下发到MCU的信号
  ]
}
```

重新运行代码生成工具 – 自动更新 x_dom_can_rt_gen_uds/ 下的 cfg.h/cfg.cpp/sigif.h/sigif.cpp。生成后你会得到类似 COM 那样的 SigIf_Get_VDC_PowerMode() 接口。

步骤2: 打开编译开关

```
# fawhq_e001_10_config.cmake
DOPTION(CONFIG_X_DOM_CAN_RT_UDS_V4 ON) # 从 OFF 改为 ON
```

步骤3: 确认 CMake 传递链自动生效

consys_bf.cmake 第181行会自动把 CAN comif 静态库加入 UDS v4 的链接列表：

```
if(CONFIG X_DOM_CAN_RT_UDS_V4)
    # 将 x_dom_can_rt_gen_lib_uds_fawhq_e001_l0_v4 加入 product_static_lib_list_uds_v4
endif()
```

而 x_dom_can_rt_gen_uds/CMakeLists.txt 会自动把新生成的 cfg.cpp + sigif.cpp + SDK 层 (sig_route_soc.cpp, x_dom_can_rt_soc.cpp) 编译进去。

需要注意的是

UDS v6 的 CAN comif 库会额外编译 COM 的生成源文件 (x_dom_can_rt_gen_com_src_list)，因为 UDS v6 接收 COM 通过 DCMS topic 发来的二进制数据需要知道 COM 侧的结构体布局。UDS v4 的模式取决于你 JSON 中 read_mcu_rx_sigs_cfg 的信号来源——如果信号来自 COM 域转发，也需要类似处理。

另一条路：P2P 直连（不经过 COM 域）

代码中预留了 P2P 通道 (x_dom_can_rt_internal.h:20-21)：

```
#define XDOMCANRT_P2P_READ_SIGS_MCU_TO_AD_TOPIC_NAME "/x_dom_can_rt/read_sigs/mcu_to_ad_app"
#define XDOMCANRT_P2P_WRITE_SIGS_AD_TO_MCU_TOPIC_NAME "/x_dom_can_rt/write_sigs/ad_app_to_mcu"
```

目前只有 AD 域的 P2P，没有 UDS 的。如果 UDS 需要绕开 COM 直连 MCU，需要新增类似 topic。

Update Todos

- ✓ 追溯 config.cmake 宏 → CMakeLists.txt 的传递链
- ✓ 分析 libuds_v4.so 的完整 SO 依赖关系
- ✓ 分析 CAN 信号接入 UDS 的依赖引入
- ✎ 梳理 comif ↔ COM 链路的完整调用链

Thought for 7s >

Read sig_route_soc.cpp

Thought for 34s >

问题4：comif 与正常 COM 链路的完整调用链

整个 comif 基于 DCMS topic（发布/订阅 + 客户端/服务器）实现跨域通信。核心机制是：每个域注册为 DCMS 的“node”，通过 topic 收发数据。

4.1 初始化阶段：各域注册 topic

入口是 x_dom_can_rt_soc.cpp:82-103，根据进程名走不同分支：

```
x_dom_can_rt_init("com")    - node="com_app_comif"    -> 注册8个topic
x_dom_can_rt_init("doipv4") - node="uds_app_comif"    -> 注册2个topic
x_dom_can_rt_init("ad")     - node="ad_app_comif"     -> 注册对应topic
```

COM 域注册的 8 个 topic（来自 x_dom_can_rt_dcms_config_com_app.h）：

topic	方向	角色
/x_dom_can_rt/mcu_tx_rx_msgs	MCU → COM	COM=server
/x_dom_can_rt/write_to_mcu_msgs	COM → MCU	COM=client
/x_dom_can_rt/write_sigs	UDS/AD → COM	COM=server
/x_dom_can_rt/read_sigs/dji_uds_service	COM → UDS	COM=client
/x_dom_can_rt/read_sigs/dji_application	COM → AD	COM=client
/x_dom_can_rt/read_sigs/dji_application/ps	COM → AD (push)	COM=pub

UDS 域注册的 2 个 topic（来自 x_dom_can_rt_dcms_config_uds_app.h）：

topic	方向	角色
/x_dom_can_rt/write_sigs	UDS → COM	UDS=client
/x_dom_can_rt/read_sigs/dji_uds_service	COM → UDS	UDS=server

4.2 下行链路：MCU → COM → UDS

```
MCU (AUTOSAR CanIf)
├─ dcms_mcu_topic_send()
├─ topic: "/x_dom_can_rt/mcu_tx_rx_msgs"
└─ ▼
   ┌─ COM 域 (com_app_comif node)
   └─ ① msg_route_mcu_tx_rx_msgs_topic_cbk() [msg_route_soc:490]
      ┌─ 遍历收到的 CAN 帧
      └─ push 到 mcu_can_msg_queue
         └─ 立即调用 msg_route_mcu_tx_rx_can_msgs_proc() [msg_route_soc:503]
            └─ ② msg_route_mcu_tx_rx_can_msgs_proc() [msg_route_soc:93]
               ┌─ 从 queue 中逐条取出 CAN 帧
               └─ 查 msg_route_mcu_tx_rx_msgs_map (channel, can_id)
                  └─ CanMsg::set_data(data, dlc, status) - 写入信号缓存
                     └─ CanMsg::add_und_cnt() - 更新帧计数
```



```
CanMsg::add_upd_cnt(); // 更新帧计数
}

③ #if XDOMCANRT_COM_REALTIME_FWD_SIGS_TO_APP_EN [msg_route_soc:507]
    sig_route_send_mcu_tx_rx_sigs() // 实时转发!
    (此函数在 COM 的 x_dom_can_rt_cfg.cpp 中, 自动生成)

    遍历所有 MCU TX/RX 信号
    SigHdl::signal_if_get(), 从 CanMsg 缓存读取物理值
    填充结构体 → dcms_mcu_topic_send()

    topic: "/x_dom_can_rt/read_sigs/dji_uds_service"
    (COM=client → UDS=server)

    topic: "/x_dom_can_rt/read_sigs/dji_application"
    (COM=client → AD=server)
}

UDS 域 (uds_app_comif node)

④ sig_route_mcu_rx_sigs_topic_cbk() [sig_route_soc:68]
    (仅当 XDOMCANRT_DJI_COM_SERVICE_EN==0)
    sig_route_update_mcu_tx_rx_sigs_from_com(data, len)
    [x_dom_can_rt_cfg.cpp:53, 自动生成]
    校验 hash_value
    memcpy 到本地结构体缓存
    设 update_flag = true

⑤ 上层代码通过 SigIf_Get_XXX() 读取信号物理值
    (当前 e001_10 UDS 的 CAN 信号为空, 所以这些函数不存在)
```

4.3 上行链路: UDS → COM → MCU

```
UDS 域

① sig_route_main_function() 每50ms调用 [sig_route_soc:217]
    sig_route_send_write_to_mcu_sigs_proc() [sig_route_soc:35]
    sig_route_send_write_to_mcu_sigs() [x_dom_can_rt_cfg.cpp:88]
    (仅当 XDOMCANRT_DJI_COM_SERVICE_EN==0)
    dcms_mcu_topic_send()
    topic: "/x_dom_can_rt/write_sigs"
    (UDS=client → COM=server)

COM 域

② sig_route_rcv_write_to_mcu_sigs_topic_cbk() [sig_route_soc:52]
    (仅当 XDOMCANRT_DJI_COM_SERVICE_EN==1)
    sig_route_update_write_to_mcu_msgs(data, len)
    更新 msg_route_write_to_mcu_msgs_map (CanMsg 缓存)

③ msg_route_main_function() 每50ms调用 [msg_route_soc:452]
    msg_route_write_msgs_to_mcu_proc() [msg_route_soc:241]
    按周期表从 CanMsg 缓存读取信号
    打包成 can_msg_t
    dcms_mcu_topic_send()
    topic: "/x_dom_can_rt/write_to_mcu_msgs"
    (COM=client → MCU=server)

MCU (AUTOSAR CanIf), 写入 CAN 总线
```

4.4 核心宏开关控制的两种角色

同一个 SDK 代码 (sig_route_soc.cpp) 编译进不同域时, 靠 XDOMCANRT_DJI_COM_SERVICE_EN 区分行为。这个宏在各域的 x_dom_can_rt_cfg.h (自动生成) 中定义:

宏	COM 域	UDS 域
XDOMCANRT_DJI_COM_SERVICE_EN	= 1	= 0
XDOMCANRT_READ_MCU_TX_SIGS_EN	= 1	= 0 (当前 e001_10)
XDOMCANRT_WRITE_SIGS_TO_MCU_EN	= 1	= 0

- COM (=1) : 主动从 MCU 收 CAN 帧 → 解码 → 打包 → 通过 topic 推送给 UDS/AD
- UDS/AD (=0) : 被动接收 COM 推送的信号、主动通过 topic 把写入请求发给 COM

4.5 一句话总结调用链

4.3 可总结调用链

```
MCU CAN帧
→ COM:msg_route_mcu_tx_rx_msgs_topic_cbk()
→ msg_route...proc() 解码到 CanMsg 缓存
→ sig_route_send_mcu_tx_rx_sigs() [realtime] 打包信号
→ dcms_mcu_topic_send("/read_sigs/dji_uds_service")
→ UDS:sig_route_mcu_rx_sigs_topic_cbk()
→ sig_route_update_mcu_tx_rx_sigs_from_com() 更新缓存
→ SigIf_Get_XXX() 上层读取
```

COM 和 UDS 之间没有任何动态库依赖，纯靠 DCMS topic IPC 通信。

Update Todos

- ✓ 追溯 config.cmake 宏 → CMakeLists.txt 的传递链
- ✓ 分析 libuds_v4.so 的完整 SO 依赖关系
- ✓ 分析 CAN 信号接入 UDS 的依赖引入方式
- ✓ 梳理 comif → COM 链路的完整调用链

Thought for 1s >

以上4个问题全部回答完毕。核心要点：

- 宏传递链：** doption() = set() → CMake 变量 → if() 条件编译 → 决定哪些静态库进 \${product_static_lib_list_uds_v4} → 最终 --whole-archive 打进 libuds_v4.so
- SO 依赖：** libuds_v4.so 不依赖 lib_com.so，两者编译期和链接期完全独立，运行时通过 DCMS topic IPC 松耦合通信
- CAN 信号接入：** 改 JSON 配置 → 重新生成代码 → CONFIG_X_DOM_CAN_RT_UDS_V4 = ON → 自动链接
- comif 调用链：** MCU CAN 帧 → COM 域 msg_route 解码缓存 → sig_route_send 实时/周期转发 → DCMS topic → UDS 域 sig_route_mcu_rx_sigs_topic_cbk 接收 → SigIf_Get 上层读取。整个过程 COM 域是唯一的 MCU CAN 网关。

```
</ consys_bf.cmake#198-198
```

x_dom_can_rt_gen_lib_com、x_dom_someip_rt_gen_lib_com、x_dom_can_rt_gen_lib_uds_v4、x_dom_someip_rt_gen_lib_uds_v4 这些库的编译依赖了哪些文件，调用关系是怎样的

Thought for 4s >

Bash Read all 4 CMakeLists.txt and list source files in each gen directory

```
IN echo "=== COM CAN ===" && cat /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/fawhq_e001_10/proxy/x_dom_can_rt_gen_com/CMakeLists.txt && echo

OUT === COM CAN ===
message(STATUS "x_dom_can_rt_gen:${PRODUCT_NAME}")
```

Thought for 2s >

Bash List all source files for all 4 gen libs and SDK sources

```
IN echo "=== COM CAN gen files ===" && ls -la /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/fawhq_e001_10/proxy/x_dom_can_rt_gen_com/ && echo

OUT === COM CAN gen files ===
总计 332
drwxrwxr-x 2 c-ziming.jiang c-ziming.jiang 4096 5月 7 15:38 .
```

Thought for 13s >

Bash Read COM CAN cfg.h macros section

```
IN head -60 /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/fawhq_e001_10/proxy/x_dom_can_rt_gen_com/x_dom_can_rt_cfg.h

OUT // Tool-generated code(x_dom_can_rt_fawhq_e001_10.json), do not modify manually.
// 2026-04-27 18:26:13
// Tool-version = v0.17, Tool-hash-value = 178a0d03b530dd073c36f33ac21ff9b
```

Thought for 2s >

Read x_dom_can_rt_sigif.cpp

Thought for 3s >

Read x_dom_can_rt_cfg.cpp

Thought for 2s >

Read x_dom_someip_rt_sigif_get.cpp

Thought for 3s >

Read x_dom_someip_rt_cfg.cpp

Thought for 3s >

Bash Read COM SOMEIP cfg.cpp to see topic config and how data is sent to UDS

```
IN grep -n "topic_node_name\|TOPIC_NODE_NAME\|svc_route\|x_dom_someip_rt_set\|com_to_app" /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/fawhq_

OUT 165:const std::map<uint32_t, x_dom_someip_rt::SvcItemInfo*> svc_route_rcv_svc_info_map =
    298:static x_dom_someip_rt::NodeRecvInfo svc_route_rcv_svc_info_ad_app =
    425:    "/x_dom_someip_rt/com_to_app/ad_app"
```

Thought for 3s >

Read x_dom_someip_rt_cfg.cpp (lines:426-450)

Thought for 2s >

现在我可以给出完整的文件级依赖和调用关系图。

四个 comif 静态库的完整文件组成与调用关系

一、文件组成总览

四个库的源文件 = 自动生成代码 + SDK通用层

库1: x_dom_can_rt_gen_lib_com (COM CAN comif, 280KB cfg.cpp)

来源	文件	大小	作用
gen	x_dom_can_rt_cfg.h	28KB	宏开关 + SigHdl声明 + 信号结构体定义
gen	x_dom_can_rt_cfg.cpp	280KB	CanMsg对象(37个) + msg_route map + SigHdl对象(60+) + sig_route_send_mcu_tx_rx_sigs() 实现 + sig_route_update_write_to_mcu_msgs() 实现 + shell命令
gen	x_dom_can_rt_sigif.h	3.5KB	SigIf_Get_XXX() 函数声明 (60+个信号)
gen	x_dom_can_rt_sigif.cpp	8.7KB	SigIf_Get_XXX() 实现 → 调用对应 SigHdl::signal_if_get()
SDK	sig_route_soc.cpp	7.3KB	topic回调注册 + sig_route_main_function()
SDK	x_dom_can_rt_soc.cpp	6.6KB	x_dom_can_rt_init() 入口 + 周期任务调度
SDK	msg_route_soc.cpp	21.9KB	msg_route_mcu_tx_rx_can_msgs_proc() + topic回调
SDK	p2p_parser.cpp	9.5KB	P2P直连解析

通过 aux_source_directory(\${X_DOM_CAN_RT_SRC} ...) 把整个 SDK 目录的4个 cpp 全部扫入

库2: x_dom_someip_rt_gen_lib_com (COM SOME/IP comif, 1.4MB protocol)

来源	文件	大小	作用
gen	x_dom_someip_rt_cfg.h	57KB	服务ID宏定义 + 服务映射表声明
gen	x_dom_someip_rt_cfg.cpp	144KB	svc_route_rcv_svc_info_uds_app (指定转发到UDS的topic "/x_dom_someip_rt/com_to_app/uds_app") + x_dom_someip_rt_prod_cfg()
gen	x_dom_someip_rt_protocol.hpp	232KB	序列化/反序列化struct定义
gen	x_dom_someip_rt_protocol_0~2.cpp	1.5MB	三个通道的协议编解码实现
gen	x_dom_someip_rt_sigif.h/cpp	~4KB	SigIf_XXX_Get/Set/SetCallBack
gen	x_dom_someip_rt_dbg.*	~168KB	调试shell命令

库3: x_dom_can_rt_gen_lib_uds_v6 (UDS CAN comif, 当前OFF)

来源	文件	大小	作用
gen	x_dom_can_rt_cfg.h/cpp	2.5KB+4.8KB	空的信号结构体 (hash+name)
gen	x_dom_someip_rt_sigif.h/cpp	385B+610B	空的 SigIf 接口
gen (COM)	x_dom_can_rt_gen_com/*.h	—	COM域的全部生成代码
SDK	sig_route_soc.cpp	7.3KB	topic回调(接收端)
SDK	x_dom_can_rt_soc.cpp	6.6KB	x_dom_can_rt_init("doipv4")

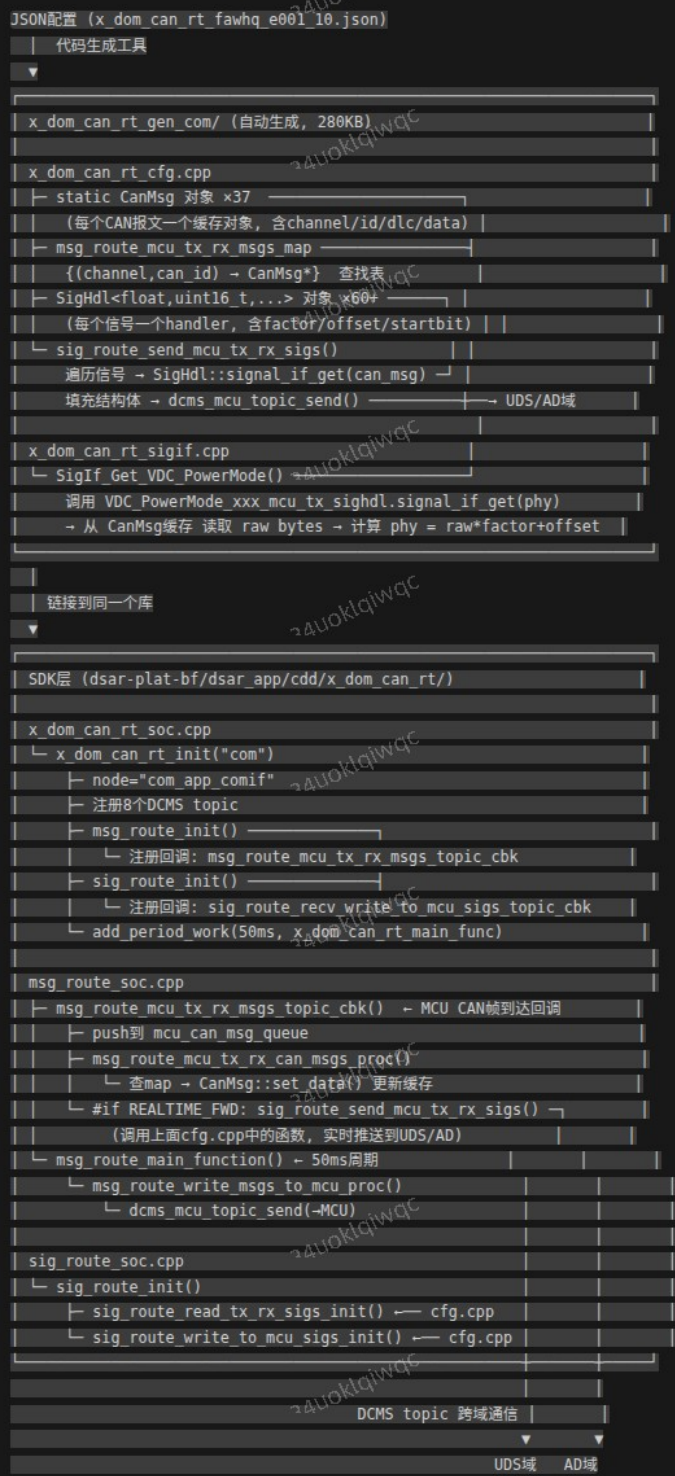
注意: CMakeLists 中库名是 _v6, 不是 _v4, 当前产品 CONFIG_X_DOM_CAN_RT_UDS_V6=OFF, 所以此库虽然被编译但不链接进任何 .so

库4: x_dom_someip_rt_gen_lib_uds_v4 (UDS SOME/IP comif, ON)

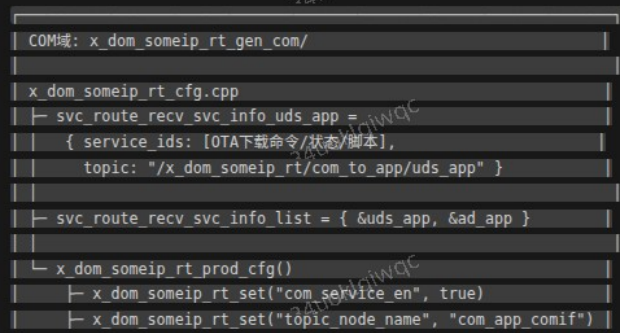
来源	文件	大小	作用
gen	x_dom_someip_rt_cfg.h	25KB	74+服务ID宏
gen	x_dom_someip_rt_cfg.cpp	6.5KB	svc_route_write_someip_init() 注册 DCMS topic "/x_dom_someip_rt/com_to_app/uds_app" (UDS=sub) + x_dom_someip_rt_prod_cfg()
gen	x_dom_someip_rt_protocol.hpp	6.3KB	协议struct定义
gen	x_dom_someip_rt_protocol_0.cpp	35KB	协议编解码
gen	x_dom_someip_rt_sigif_get.cpp	8.9KB	SigIf_OnRecvData() → 按service_id分发回调
gen	x_dom_someip_rt_sigif_set.cpp	3.5KB	SigIf_XXX_Set() 发送端
gen	x_dom_someip_rt_sigif.h	2.4KB	SigIf_XXX_Get/Set/SetCallBack 声明
gen	x_dom_someip_rt_dbg.*	8KB	调试命令

二、调用关系图

CAN comif 链路 (COM 域)



SOME/IP comif 链路 (COM → UDS)



```
└─ 注入 rcv_svc_info_list、send_svc_list 等
运行:
运行: 当COM从AUTOSAR收到SOME/IP服务数据
└─ x_dom_someip_rt SDK匹配 service_id
└─ 查 svc_route_rcv_svc_info_list 找到对应的 node
└─ dcms_mcu_topic_send(topic="/x_dom_someip_rt/com_to_app/uds_app")
DCMS topic: "/x_dom_someip_rt/com_to_app/uds_app"
UDS域: x_dom_someip_rt_gen_uds_v4/
x_dom_someip_rt_cfg.cpp
└─ svc_route_write_someip_init()
└─ dcms_mcu_topic_setup_callback(
    topic="/x_dom_someip_rt/com_to_app/uds_app",
    cbk=数据到达回调)
x_dom_someip_rt_prod_cfg()
└─ x_dom_someip_rt_set("app_service_en", true)
└─ x_dom_someip_rt_set("topic_node_name", "uds_app_comif")
└─ 注入 rcv/send_svc_info_map
运行时: DCMS回调收到COM发来的SOME/IP数据
└─ x_dom_someip_rt SDK 反序列化
└─ SigIf_OnRecvData(service_id) [sigif_get.cpp]
└─ switch(service_id)
└─ case OTA下载命令:
    SigIf_XXX_Get(&data) → 反序列化
    → 回调 SigIf_XXX_Cb(&data, ctx) // 通知上层
└─ case OTA下载状态: ...
x_dom_someip_rt_sigif_set.cpp (UDS主动发送方向)
└─ SigIf_XXX_Set(&data)
└─ 序列化 → x_dom_someip_rt SDK
└─ dcms_mcu_topic_send(topic="/x_dom_someip_rt/...")
└─ COM域接收 → AUTOSAR SOME/IP Stack → 发出
```

三、四个库的关键差异

	COM CAN	COM SOMIP	UDS CAN v6	UDS SOMIP v4
node名	com_app_comif	com_app_comif	uds_app_comif	uds_app_comif
角色	生产者(MCU网关)	生产者(中继)	消费者	消费者
包含SDK	sig_route_soc+msg_route_soc+x_dom_can_rt_soc+... (自带SDK)	(自带SDK)	sig_route_soc+x_dom_can_rt_soc+... (自带SDK)	(自带SDK)
包含COM gen	否(自己就是COM)	否	是(cfg.cpp+cigif.cpp全量)	否
XDOMCANRT_DJI_COM_SERVICE_EN	1	—	0	—
数据方向	MCU → 缓存 → topic → 各域	AUTOSAR → topic → 各域	topic → 回调 → 缓存	topic → 回调 → SigIf_OnRecvData()
对端topic	write_sigs 收, read_sigs/uds 发	com_to_app/uds_app 发	read_sigs/uds 收	com_to_app/uds_app 收
当前状态	ON, 打入 lib*_com.so	ON, 打入 lib*_com.so	OFF	ON, 打入 libuds_v4.so

四、一句话总结调用链

CAN路径: SigIf_Get_XXX() → SigHdl::signal_if_get() → 读 CanMsg::data_buf → 解码(factor/offset/startbit) → 返回物理值				
SOME/IP路径: COM域 AUTOSAR收包 → NodeRecvInfo topic路由 → DCMS → UDS回调 → SigIf_OnRecvData() → 反序列化 → 上层回调				
当前状态	ON, 打入 lib*_com.so	ON, 打入 lib*_com.so	OFF	ON, 打入 libuds_v4.so

四、一句话总结调用链

• CAN路径: SigIf_Get_XXX() → SigHdl::signal_if_get() → 读 CanMsg::data_buf → 解码(factor/offset/startbit) → 返回物理值	• SOME/IP路径: COM域 AUTOSAR收包 → NodeRecvInfo topic路由 → DCMS → UDS回调 → SigIf_OnRecvData() → 反序列化 → 上层回调
--	--